A Reinforcement Learning–Guided General Variable Neighborhood Search for the Traveling Salesman Problem

Karina Assini Andreatta[1 [0009−0001−7805−2594]], Maria Claudia Silva Boeres[1 [0000−0001−9801−2410]], and Renato Elias Nunes de Moraes[1 [0000−0001−5538−0504]]

Federal University of Espírito Santo (UFES): Av. Fernando Ferrari 514, Goiabeiras, Vitória – ES, 29075-910, Brazil
karinassini@gmail.com
https://www.ufes.br

Abstract. This paper proposes RL-GVNS, a reinforcement learning–guided General Variable Neighborhood Search framework for the Traveling Salesman Problem (TSP), which introduces adaptivity in both the construction and intensification phases. The proposed RL-GVNS extends the classical GVNS by integrating a multi-agent reinforcement learning (MARL) initializer and a reinforcement learning–driven Variable Neighborhood Descent, where neighborhood operators are selected online using Q-learning. Unlike standard GVNS, which relies on a fixed ordering of local search operators, RL-GVNS dynamically adapts its search behavior based on performance feedback, enabling an effective response to non-stationary search dynamics. Although all evaluated configurations, combining classical or MARL-based initialization with standard GVNS or RL-GVNS, perform similarly in small instances, learning-enhanced variants show superior robustness as instance size increases. In particular, the combined MARL + RL-GVNS configuration achieves lower average gaps with respect to the best-known solution, higher best-known solution hit rates, and, in most cases, comparable or reduced execution times relative to non-adaptive GVNS baselines.

Keywords: General Variable Neighborhood Search · Reinforcement Learning · Traveling Salesman Problem.

1 Introduction

The Traveling Salesman Problem (TSP) is a classical benchmark in combinatorial optimization. Given the pairwise travel costs between cities, the goal is to find a minimum-cost Hamiltonian tour. Although simply defined, TSP is NP-hard, limiting the applicability of exact methods to medium- and large-scale instances [20]. The TSP exemplifies a broader class of complex combinatorial optimization problems in which the search space is discrete and characterized by a large number of local optima. As the size of the problem increases, exact

methods tend to become computationally expensive and, in some cases, impractical. This limitation motivates the use of metaheuristic approaches, which yield high-quality solutions at practical computational time [24].

Among these, Variable Neighborhood Search (VNS), proposed in [23], leverages this trade-off by relying on a systematic exploration of multiple neighborhood structures. The underlying principle of VNS is that a solution that is locally optimal with respect to one neighborhood structure may cease to be optimal when evaluated under a different neighborhood. To exploit this property, the method alternates between diversification and intensification by changing the neighborhood during the search process. Diversification is performed through a shaking phase, which applies controlled perturbations to move the search away from the current solution, while intensification is achieved by applying local search procedures within a selected neighborhood to refine the solution and reach a local optimum [19].

In General Variable Neighborhood Search (GVNS), intensification is commonly implemented through Variable Neighborhood Descent (VND) [12]. However, the fixed ordering of local search operators in standard VND leads to static search behavior, motivating adaptive neighborhood selection strategies. Although GVNS has been successfully applied to computationally hard optimization problems that arise in practical settings, such as routing, scheduling, and network design [7,22,32,18,16], recent studies have increasingly explored the integration of machine learning techniques to guide metaheuristic decisions, including promising results obtained through reinforcement learning–based operator adaptation [27,31,17].

Despite recent progress, most approaches focus on population-based or evolutionary frameworks [15,2,28], or address individual search components using static or weakly adaptive rules. Neighborhood-based metaheuristics for the TSP rely mainly on fixed operator selection [7,32,14], limiting their adaptability to the evolving search space. This limitation becomes particularly relevant in the context of the TSP, whose search dynamics is inherently non-stationary. Operators that contribute to improving the solution in the early stages of the search may lose relevance as the algorithm converges to high-quality local optima [19].

Table 1 shows that many approaches incorporate adaptability at different stages of the search process. In contrast, the proposed work introduces adaptability primarily in the local search step, using reinforcement learning to dynamically guide the order in which operators are applied within the VND framework through an Adaptive Operator Selection (AOS) mechanism [9]. By learning the characteristics and effectiveness of operators throughout the search, the method enhances intensification while preserving robustness, distinguishing it from GVNS-based approaches that rely on fixed operator schemes. In addition, the shaking phase has been redesigned to avoid static behavior. The shaking process contains an element of adaptability in the sense that, rather than relying on a single fixed perturbation, multiple shaking operators are employed within each neighborhood. Although this mechanism does not constitute operator se-

lection in the learning-based sense, it introduces a complementary approach that improves diversification and reduces the sensitivity to manual parameter tuning.

Table 1. Comparative analysis of related works. Columns indicate: (1) use of VNS or GVNS; (2) population-based approach; (3) explicit parameter calibration or control; (4) adaptive operator selection (AOS); (5) adaptive shaking mechanism; (6) availability of open-source implementation.

Author	**(1)**	**(2)**	**(3)**	**(4)**	**(5)**	**(6)**
da Silva et al. (2010) [7]	✓	✗	–	✗	✗	–
Queiroz dos Santos et al. (2014) [27]	✓	✗	–	✓	✗	–
Todosijevic et al. (2016) [31]	✓	✗	–	✓	✗	–
Menendez et al. (2017) [22]	✓	✗	–	✗	✗	–
Venkatesh et al. (2018) [32]	✓	✗	–	✗	✗	–
Hore et al. (2018) [14]	✓	✗	–	✗	✗	–
Alipour et al. (2018) [2]	✗	✓	–	✗	✗	–
Karakostas et al. (2019) [18]	✓	✗	Partial	✗	✗	–
Karakostas et al. (2020) [16]	✓	✗	–	✗	✗	–
Karakostas et al. (2022) [17]	✓	✗	–	✓	✓	–
Ruan et al. (2024) [28]	✗	✓	–	✗	✗	–
Jeong et al. (2025) [15]	✗	✓	–	✓	✗	–
This work	✓	✗	✓	✓	✓	✓

The remainder of the paper is structured as follows. Section 2 introduces the TSP and reviews the GVNS framework. Section 3 describes the proposed RL-GVNS method, including adaptive initialization, shaking strategy, and local search driven by reinforcement learning. Computational experiments and results are presented in Section 4. Finally, Section 5 summarizes the main findings and discusses directions for future work.

2 Preliminaries

2.1 Traveling Salesman Problem (TSP)

The Traveling Salesman Problem (TSP) is a canonical problem in combinatorial optimization and a standard benchmark for heuristic and metaheuristic algorithms [24]. In this work, the symmetric variant is considered and defined on a complete weighted graph $G = (V, E)$, where $V = \{1, 2, \ldots, n\}$ represents the set of cities and each edge $(i, j) \in E$ is associated with a non-negative travel cost c_{ij}, with $c_{ij} = c_{ji}$ for all $i, j \in V$.

A feasible solution is a Hamiltonian cycle, represented by a permutation $\pi = (\pi_1, \pi_2, \ldots, \pi_n)$ of cities, where π_k denotes the city visited at position k in the tour. In closed tour representations, the starting city may appear duplicated at the end of the sequence. However, the size of the problem n is defined as

the number of distinct cities in the tour after removing the duplicated terminal node in the closed representation. The objective is to find a permutation that minimizes the total travel cost along the cycle, including the return to the starting city. Since the size of the search space increases factorially with n, exhaustive enumeration quickly becomes infeasible, motivating the use of heuristic and metaheuristic approaches that explore neighborhoods defined over permutations to obtain high-quality solutions.

2.2 General Variable Neighborhood Search (GVNS)

The GVNS framework operates on a predefined set of neighborhoods and combines systematic diversification with a deterministic intensification mechanism based on VND [12]. Within GVNS, the VND procedure acts as the main local search phase, where neighborhoods are explored sequentially according to a fixed order. Whenever an improvement move is identified, the incumbent solution is updated and the search restarts from the first neighborhood. The procedure advances to the next neighborhood until no further improvement can be obtained, at which point the solution is considered locally optimal with respect to the available structures.

In GVNS, this intensification step is integrated into an outer search loop that alternates between shaking and local improvement, enabling controlled exploration of increasingly distant regions of the solution space. Additional details on GVNS are provided in [12,11].

2.3 Adaptive Operator Selection (AOS)

Adaptive Operator Selection (AOS) [9] provides a control layer that regulates the choice of search operators according to their observed contribution during the search. Rather than prescribing a specific learning strategy, it frames operator selection itself as a decision problem, delegating the underlying adaptation mechanism to the method in use. Within the machine learning in metaheuristics (ML-in-MH) paradigm, reinforcement learning has therefore been adopted as a practical means of implementing AOS in search processes that evolve over time [19].

In permutation-based problems such as the TSP, adaptive operator selection has been mainly investigated in population-oriented metaheuristics, including evolutionary algorithms and adaptive large neighborhood search [27,2]. In contrast, trajectory-based approaches, such as VNS and GVNS, typically rely on predefined operator schedules throughout the search process.

Despite the extensive literature on neighborhood design for the TSP, the dynamic selection of operators during the search has received comparatively limited attention, particularly in neighborhood-based metaheuristics based on single-solution.

Reinforcement Learning (RL) [29] is a method in which decisions are adapted over time based on feedback obtained from previous results. Rather than relying on predefined rules, the decision process progressively favors actions that have previously led to improvements, while discouraging those associated with ineffective or detrimental moves. In the context of neighborhood-based metaheuristics, this allows the selection of local search operators to be guided by their observed contribution to solution quality during the search.

From this perspective, operator selection can be interpreted as an adaptive decision process in which local search operators correspond to actions, and performance feedback provides the learning signal used to guide subsequent choices. The contribution of each operator is evaluated through reward functions that reflect its impact on search progress, allowing the learning of a policy that accounts for delayed effects and variations in effectiveness along the search trajectory.

Within this formulation, AOS can be viewed as an online control problem in which operator utilities are updated from observed rewards, enabling the selection of moves that remain effective under changing search conditions. As described in [9], this adaptation cycle involves performance evaluation, reward attribution, credit assignment, and operator selection. In the proposed method, credit assignment and selection are implemented through a Q-learning strategy [34].

Q-learning Q-learning is an RL method that assigns a value to each possible action according to its expected outcome in a given state. These values are updated iteratively from the observed rewards, allowing the selection of actions that lead to improved results over time without requiring an explicit model of the underlying process.

The learning process is commonly formalized as a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ the action set, P the transition probabilities, and R the reward function. At each decision step t, the agent observes the current state $s_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}$, receives a reward r_t, and transitions to a new state s_{t+1} [33].

Q-learning estimates the action–value function $Q(s, a)$, representing the expected discounted cumulative reward of selecting action a in state s and following the learned policy thereafter. The estimates are updated iteratively according to the Bellman optimality:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a \in \mathcal{A}} Q(s_{t+1}, a) - Q(s_t, a_t) \right], \tag{1}$$

where $\alpha \in (0.0, 1.0]$ is the learning rate and $\gamma \in [0.0, 1.0)$ is the discount factor.

Under standard conditions, that is, bounded rewards, satisfaction of the Markov property, and sufficient exploration of state–action pairs, Q-learning is guaranteed to converge with probability one to the optimal action–value function [29,33].

3 Proposed Algorithm

The proposed algorithm extends GVNS with reinforcement learning–driven local search, using continuous credit assignment to adapt operator selection to the evolving search landscape. The Algorithm 1 receives as input an initialization method `InitMethod`, an iteration limit $I_{\max}$, and a maximum neighborhood index $k_{\max}$. The initialization method controls how the initial tour is constructed and can assume one of two alternatives: a classical restricted candidate list nearest-neighbor heuristic (RCL) or a learning-based multi-agent reinforcement learning initializer (MARL), both described in detail in Section 3.1.

The algorithm starts by generating an initial tour x according to the selected initialization strategy and computing its objective value $c = f(x)$ (lines 1–2). The search then proceeds within an outer loop limited to $I_{\max}$ iterations (lines 3-17). At the beginning of each iteration, the neighborhood index k is reset to 1 (line 4), and neighborhoods of increasing size are explored within an inner loop (lines 5–16).

For a given neighborhood size k, a shaken solution x' is generated to promote diversification (line 6) and subsequently intensified through a local search procedure driven by reinforcement learning, which returns an improved candidate solution x'' and its associated cost c'' (lines 7–8). If the candidate solution improves on the incumbent ($c'' < c$, line 9), the incumbent solution is updated and the neighborhood index is reset to $k = 1$, restarting the search from the smallest neighborhood to intensify the exploration around the improved tour (lines 10–12). Otherwise, the neighborhood index is incremented to explore larger neighborhoods and increase diversification (line 14). After completing all iterations, the algorithm returns the best tour found during the search.

3.1 Initialization

The quality of the initial solution plays a critical role in the performance of trajectory-based metaheuristics for the TSP, as it directly influences the regions of the search space explored in the early stages of the algorithm [26]. Classical construction procedures for the TSP often generate an initial tour through deterministic or partially randomized heuristics such as nearest-neighbor variants or restricted candidate list (RCL) methods [4]. These approaches build a solution sequentially based on locally available information, without revisiting earlier construction decisions once the tour has been formed.

In this work, two initialization procedures are employed. The first corresponds to a nearest-neighbor heuristic augmented with an RCL mechanism, which introduces controlled variability in the selection of candidate cities at each construction step [4]. The second adopts a learning-based strategy inspired by multi-agent reinforcement learning (MARL) [1], where the tour is assembled incrementally using action-value estimates that are updated throughout the construction process according to the evolving partial solution.

The learning mechanism is adapted to the single-solution setting of GVNS, rather than the population-based framework adopted in [1], with the objective

Algorithm 1 GVNS with RL-driven Local Search

Require: initialization method `InitMethod`, iteration limit $I_{\max}$, neighborhood limit $k_{\max}$

$x \leftarrow$ INITIALIZE(`InitMethod`)
$c \leftarrow f(x)$
for $i = 1$ to $I_{\max}$ **do**
  $k \leftarrow 1$
  while $k \leq k_{\max}$ **do**
    $x' \leftarrow$ SHAKE(x, k)
    $c' \leftarrow f(x')$
    $(x'', c'') \leftarrow$ RLLOCALSEARCH(x', c')
    if $c'' < c$ **then**
      $x \leftarrow x''$
      $c \leftarrow c''$
      $k \leftarrow 1$ ▷ restart from the first neighborhood
    else
      $k \leftarrow k + 1$
    end if
  end while
end for
return x

of producing a small set of high-quality candidate tours and a strong starting solution. Unlike MARL-based construction methods that update values only after complete tour evaluations, the proposed initializer employs a Q-learning formulation grounded in the Bellman optimality principle (Equation 1). Incorporating both immediate and delayed rewards allows value estimates to account for the influence of early construction choices on subsequent search behavior, supporting consistent updates during tour generation.

Both initialization procedures are implemented in GVNS via the parameter `InitMethod` in Algorithm 1, so that their effects can be examined under the same search configuration.

3.2 Adaptive Shaking Strategy

In GVNS, diversification is achieved through a shaking phase that applies random perturbations to the incumbent solution, allowing transitions to neighboring regions of the search space [11]. In the proposed adaptive shaking formulation, the neighborhood index k is associated with the intensity of the perturbation by defining, at each shaking level, a set of admissible operators of increasing strength from which a move is randomly sampled.

The magnitude of the applied perturbation is controlled through operator parameters expressed as bounded functions of the size of the instance n. Consequently, the shaking phase operates on a set of established TSP perturbation operators, including segment relocations, edge exchanges, non-sequential pertur-

bations, and block-based rearrangements, whose parameters scale with problem size.

In particular, Or-opt relocations [3], k-exchange moves [21], and cross-exchange operations [30] are applied using intensity-dependent parameterizations, while parameter-free operators such as double-bridge move [5,13] are employed at higher shaking levels. Additional variability is introduced through segment shuffling and block recombination mechanisms [25].

For segment-based operators, such as Or-opt [3] and cross-exchange [30], segment lengths are defined as bounded fractions of n, clipped between predefined minimum and maximum values. As a result, lower shaking levels modify short segments even in large instances, whereas higher levels operate on proportionally larger portions of the tour. Similarly, for exchange-based operators (e.g., k-exchange [21]), the number of exchanged nodes increases with n, while non-parametric operators such as double-bridge are applied without modification [5,13].

Formally, the adaptive shaking procedure (line 6 of Algorithm 1) is summarized in Algorithm 2. At each shake step, the neighborhood $\mathcal{N}_k(x)$ is implicitly defined by a set of perturbation operators $\mathcal{O}_k$ associated with the shaking level k (lines 3–11), whose parameters are scaled according to the size of the instance n (lines 1–2). The perturbation is then obtained by sampling an operator from $\mathcal{O}_k$ and applying it to the incumbent solution (line 14), with repeated attempts allowed to avoid trivial modifications (lines 12–18). The set of operators $\mathcal{O}_k$ is indexed by k to induce progressively stronger perturbations, ranging from mild structural changes to larger-scale tour rearrangements [5,13].

3.3 RL-driven Local Search Procedure

The RLLocalSearch procedure implements the intensification phase of the proposed GVNS framework (line 8 of Algorithm 1). Starting from the shaken tour x', its objective is to exploit the local structure of the solution and produce an improved tour by sequentially applying local search operators. The selection of these operators is guided by a Q-learning mechanism combined with an ε-greedy policy, which allows adaptive control of the intensification process based on the observed effectiveness of operator transitions.

The local search operates on a fixed set of neighborhood operators

$$\mathcal{O} = \begin{Bmatrix} \text{TwoOpt [8]}, & \text{OneMoveInsertion [10]}, \\ \text{ThreeOpt [6]}, & \text{DoubleBridge [13]} \end{Bmatrix} \tag{2}$$

which differ in neighborhood size and structural impact. The procedure follows a *first-improvement* strategy: as soon as an operator yields an improving move, the candidate solution is accepted and becomes the new incumbent, avoiding exhaustive neighborhood evaluation and favoring fast intensification.

The Algorithm 3 describes the reinforcement learning–driven local search procedure used. The procedure starts by initializing the learning components (line 1). The Q-table Q is initialized with zeros, and the exploration rate is set

Algorithm 2 Adaptive Shaking Procedure

Require: Current tour x, shaking strength k
Ensure: Perturbed tour x'
1: $n \leftarrow |x|$ ▷ number of cities in the tour representation
2: $\eta_k(n) \leftarrow$ size-dependent parameters ▷ segment lengths, exchange cardinalities
3: **if** $k \leq 1$ **then**
4: $\mathcal{O}_k \leftarrow$ {Or-opt (small), Shuffle (mild)}
5: **else if** $k = 2$ **then**
6: $\mathcal{O}_k \leftarrow$ {k-exchange (small), Cross-exchange (mild), Shuffle (medium)}
7: **else if** $k = 3$ **then**
8: $\mathcal{O}_k \leftarrow$ {Double-bridge, k-exchange (medium), Cross-exchange (medium), Block recombination (medium)}
9: **else**
10: $\mathcal{O}_k \leftarrow$ {Double-bridge, k-exchange (large), Cross-exchange (strong), Block recombination (strong), Shuffle (strong)}
11: **end if**
12: **for** a limited number of attempts **do**
13: $o \sim \mathcal{U}(\mathcal{O}_k)$
14: $x' \leftarrow o(x; \eta_k(n))$
15: **if** $x' \neq x$ **then**
16: **return** x'
17: **end if**
18: **end for**
19: **return** x

to an initial value $\varepsilon \leftarrow \varepsilon_0$, with $\varepsilon_0 \in (0.0, 1.0)$, defining the initial exploration–exploitation balance.

State–action model. Let $\mathcal{O} = \{o_1, \ldots, o_m\}$ denote the set of local search operators, and let $\tau : \mathcal{O} \to \{1, \ldots, m\}$ be an index map. The method maintains a Q-table $Q \in \mathbb{R}^{m \times m}$, where the state $s \in \{1, \ldots, m\}$ encodes the index of the previously applied operator and the action $a \in \{1, \ldots, m\}$ encodes the index of the next operator. Hence, $Q[s, a]$ estimates the utility of applying the operator a given that the previous operator was s.

After initializing the learning structures, the incumbent solution is set to the shaken tour x' and its objective value $c = f(x')$ is calculated (line 2). The set of operators is then initialized as $\mathcal{A} \leftarrow \mathcal{O}$ (line 3), an initial operator is selected from $\mathcal{A}$ (line 4), and the local search proceeds while admissible operators remain (lines 5-16). At each iteration, the selected operator is applied to the current solution, resulting in a candidate tour $\tilde{x}$ (line 6), which is subsequently evaluated to obtain its cost $\tilde{c}$ (line 7).

Algorithm 3 RL-driven Local Search

Require: Initial solution x', operator set $\mathcal{O}$, learning rate α, discount factor γ, initial exploration rate ε_0, minimum exploration $\varepsilon_{\min}$, exploration decay δ
Ensure: Improved solution x^*
1: Initialize Q-table $Q \leftarrow \mathbf{0}$ and set $\varepsilon \leftarrow \varepsilon_0$ ▷ state–action model
2: $x \leftarrow x'$, $c \leftarrow f(x)$
3: $\mathcal{A} \leftarrow \mathcal{O}$
4: $o \leftarrow \textsc{Select}(\mathcal{A}, Q, \varepsilon)$
5: **while** $\mathcal{A} \neq \emptyset$ **do**
6: $\quad \tilde{x} \leftarrow o(x)$
7: $\quad \tilde{c} = f(\tilde{x})$
8: $\quad r \leftarrow \textsc{Reward}(c, \tilde{c})$ ▷ reward design and Q-update
9: $\quad$ **if** $\tilde{c} < c$ **then**
10: $\quad\quad x \leftarrow \tilde{x}$, $c \leftarrow \tilde{c}$
11: $\quad\quad \mathcal{A} \leftarrow \mathcal{O} \setminus \{o\}$ ▷ first-improvement acceptance
12: $\quad$ **else**
13: $\quad\quad \mathcal{A} \leftarrow \mathcal{A} \setminus \{o\}$ ▷ operator pruning
14: $\quad$ **end if**
15: $\quad$ Select next operator o from $\mathcal{A}$ using ε-greedy policy ▷ ε-greedy selection
16: **end while**
17: **return** x

Reward design. After evaluating the candidate tour, the reward is calculated from the incumbent cost c and the candidate cost $\tilde{c}$ (line 8) as

$$r^+(c,\tilde{c}) = \begin{cases} \dfrac{c-\tilde{c}}{\max(c, 10^{-9})} & \text{if } \tilde{c} < c, \\ 0 & \text{otherwise} \end{cases}$$
$$r^-(c,\tilde{c}) = -\lambda \cdot \begin{cases} \dfrac{\tilde{c}-c}{\max(c, 10^{-9})} & \text{if } \tilde{c} > c, \\ 0.01 & \text{otherwise} \end{cases} \tag{3}$$

where $\lambda \geq 1$ controls the penalty scale.

Q-learning update. The Q-table is updated using the standard Q-learning Bellman rule (Equation 1) with the observed reward and the estimated value of the next state (line 8).

First-improvement acceptance and operator pruning. If the candidate solution improves with the incumbent ($\tilde{c} < c$, line 9), the improvement is immediately accepted (line 10), and the admissible operator set is reset to $\mathcal{A} \leftarrow \mathcal{O} \setminus \{o\}$ (line 11). This reset prevents the immediate reapplication of the same operator, while promoting intensification around the improved solution. If no improvement is obtained, the selected operator is removed from $\mathcal{A}$ (line 13), preventing repeated ineffective applications within the same local search call.

ε-greedy operator selection. After updating the admissible set, the next operator is selected from $\mathcal{A}$ according to an ε-greedy policy (line 15):

$$a \sim \begin{cases} \text{Uniform}(\mathcal{A}) & \text{with probability } \varepsilon, \\ \arg\max_{a \in \mathcal{A}} Q[s, a] & \text{with probability } 1 - \varepsilon \end{cases} \tag{4}$$

which balances exploitation of high-utility operator transitions and exploration of alternative moves. The procedure ends when the admissible set becomes empty, returning the best solution found.

After each call to the local search routine, the exploration rate is updated according to a multiplicative decay schedule,

$$\varepsilon \leftarrow \max(\varepsilon_{\min},\, \varepsilon \cdot \delta) \tag{5}$$

with $\delta \in (0.0, 1.0)$, gradually shifting the policy from exploration to exploitation in successive calls.

4 Computational Experiments

The experimental analysis considers two components of the GVNS framework: the initialization strategy and the search control scheme. For each component, both a classical and a reinforcement learning–based alternative are evaluated, resulting in four algorithmic configurations obtained by combining the initialization based on RCL or MARL with standard GVNS or RL-GVNS.

In the standard GVNS configuration, diversification and intensification are performed using fixed shaking operators and a classical VND procedure employing the neighborhood structures defined in Equation 2. In RL-GVNS, these static schemes are replaced by adaptive mechanisms that govern both the shaking phase and the local search, allowing the influence of reinforcement learning on the construction and subsequent search behavior to be independently assessed.

4.1 Methodology

All experiments were conducted on a single reference machine. The computational platform consisted of a MacBook Pro equipped with an Apple M3 Pro processor (11 cores) and 18 GB of RAM. All runs were executed in single-threaded mode, without parallelization. A representative subset of widely used TSPLIB (Traveling Salesman Problem Library) instances was selected to cover small, medium, and large-scale problem sizes, allowing a comprehensive evaluation of algorithmic performance. The corresponding instance characteristics and best known solution values reported in the literature are summarized in Table 2.

For each algorithm–initialization pairing, defined by a specific search algorithm combined with a particular solution initialization strategy, 30 independent runs were conducted to account for stochastic variability. All methods used identical stopping criteria based on two conditions: a maximum number of iterations and a limit on consecutive non-improving iterations. For instances with

at most 100 cities, the maximum number of iterations was set to 500 and the non-improvement limit to 200 consecutive iterations. For medium-sized instances ($100 < n < 200$) and larger instances ($n \geq 200$), these limits were reduced to 300 total iterations and 100 consecutive non-improving iterations, respectively. Performance was primarily evaluated based on the final tour length, while solution stability across independent runs was also considered.

Table 2. TSPLIB instances used in this study, including problem size and best known solution values reported in the literature.

Instance	n	Best Known
bays29	29	2020
swiss42	42	1273
gr48	48	5046
berlin52	52	7542
kroA100	100	21282
kroB100	100	22141
ch150	150	6528
si175	175	21407
pr226	226	80369
pcb442	442	50778

RL Parameters The algorithmic parameters were determined through a preliminary calibration phase inspired by the empirical evaluation adopted for the MARL component in [1]. Parameter values were determined through a preliminary calibration phase aimed at balancing solution quality, convergence behavior, and computational effort. Preference was given to configurations that exhibited stable performance across repeated runs rather than superior outcomes in isolated instances. Subsequently, minor adjustments were introduced to account for characteristics of the proposed formulation and differences in instance scale.

For the MARL-based initialization, parameter settings were selected to regulate action selection during tour construction while accommodating convergence behavior across problem sizes. In particular, the softmax temperature defined in [1] was increased to promote more selective action choices, and the number of learning iterations was reduced for larger instances in accordance with observed convergence trends.

The RLLocalSearch parameters were tuned using size-aware schedules, with faster adaptation favored for small instances and deeper, more persistent exploration adopted for medium and larger ones in order to balance convergence speed and robustness under stochastic variability. All parameter values, including learning rates, discount factors, exploration schedules, penalty scaling, iteration limits, and MARL learning iterations, are reported in Table 3.

Table 3. Parameters used in the proposed framework.

Parameter	Value(s)	Defined
Learning rate (α)	0.30 (small), 0.22 (large)	Algorithm 3, Eq. 1
Discount factor (γ)	0.50 (small), 0.80 (large)	Algorithm 3, Eq. 1
Initial exploration rate (ε_0)	0.60 (small), 0.40 (large)	Algorithm 3
Exploration decay (δ)	0.98 (small), 0.995 (large)	Algorithm 3
Minimum exploration ($\varepsilon_{\min}$)	0.05 (small), 0.02 (large)	Algorithm 3
Penalty scale (λ)	1.2 (small), 1.4 (large)	Equation 3
Failure limit	10 consecutive failures	Algorithm 3
Max. local search iterations	300 (small), 600 (large)	Algorithm 3
Softmax temperature (β)	3	Section 3.1
MARL iterations	10,000 ($n < 100$), 5,000 ($n > 100$)	Section 3.1

Analysis Table 4 reports the results obtained over 30 independent runs for each algorithm–initialization configuration. For each instance, *Best* denotes the best tour cost achieved, while *Max* and *Mean* correspond to the worst and average objective values, respectively. The *Gap (%)* represents the mean relative deviation from the best-known solution (BKS), calculated as

$$\text{Gap}(\%) = \frac{1}{R} \sum_{i=1}^{R} \frac{c_i - \text{BKS}}{\text{BKS}} \times 100, \tag{6}$$

where c_i is the cost obtained in the i-th run and $R = 30$. The *BKSH* column reports the number of runs that reach the BKS. The *IterOpt* metric indicates the average number of iterations required to reach the best solution, and *Time (s)* denotes the mean execution time. For each instance, values highlighted in bold indicate the best-performing configuration for the corresponding metric. The algorithmic configurations evaluated are: (1) RCL + GVNS, (2) MARL + GVNS, (3) RCL + RL-GVNS, and (4) MARL + RL-GVNS.

Table 4. Performance results over 30 independent runs for each algorithmic configuration.

Instance	Alg.	Best	Max	Mean	Gap (%)	BKSH	IterOpt	Time (s)
bays29	(1)	**2020**	**2020**	**2020**	**0.00**	**30**	3.13	1.17
	(2)	**2020**	**2020**	**2020**	**0.00**	**30**	3.07	1.02
	(3)	**2020**	**2020**	**2020**	**0.00**	**30**	3.60	1.41
	(4)	**2020**	**2020**	**2020**	**0.00**	**30**	**2.60**	**0.91**
swiss42	(1)	**1273**	**1273**	**1273**	**0.00**	**30**	1.90	1.12
	(2)	**1273**	**1273**	**1273**	**0.00**	**30**	0.60	**0.10**
	(3)	**1273**	**1273**	**1273**	**0.00**	**30**	1.87	1.62
	(4)	**1273**	**1273**	**1273**	**0.00**	**30**	**0.50**	0.14
gr48	(1)	**5046**	**5046**	**5046**	**0.00**	**30**	9.47	11.73
	(2)	**5046**	**5046**	**5046**	**0.00**	**30**	**6.70**	**8.37**
	(3)	**5046**	**5046**	**5046**	**0.00**	**30**	11.93	17.80
	(4)	**5046**	**5046**	**5046**	**0.00**	**30**	9.57	12.27

Continued on next page

Table 4 continued from previous page

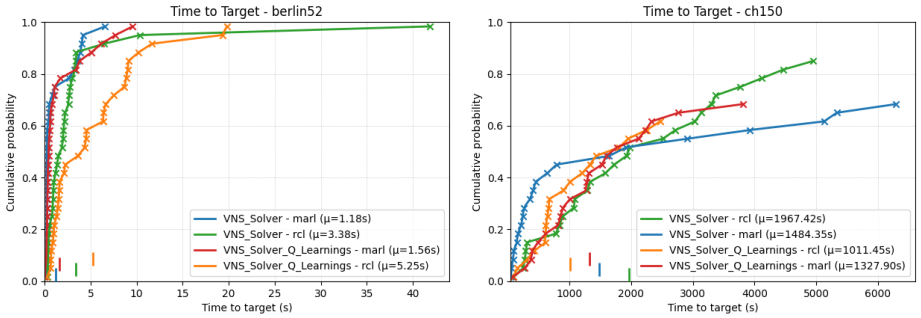
Instance	Alg.	Best	Max	Mean	Gap (%)	BKSH	IterOpt	Time (s)
berlin52	(1)	**7542**	**7542**	**7542**	**0.00**	**30**	3.07	3.38
	(2)	**7542**	**7542**	**7542**	**0.00**	**30**	1.47	**1.18**
	(3)	**7542**	**7542**	**7542**	**0.00**	**30**	3.23	5.25
	(4)	**7542**	**7542**	**7542**	**0.00**	**30**	**1.33**	1.56
kroA100	(1)	**21282**	**21282**	**21282**	**0.00**	**30**	15.87	73.07
	(2)	**21282**	**21282**	**21282**	**0.00**	**30**	8.20	**35.91**
	(3)	**21282**	**21282**	**21282**	**0.00**	**30**	17.37	120.76
	(4)	**21282**	**21282**	**21282**	**0.00**	**30**	**6.70**	48.44
kroB100	(1)	**22141**	22199	22162.27	0.10	19	**59**	683.12
	(2)	**22141**	22199	22150.67	0.04	25	75.16	**503.27**
	(3)	**22141**	22199	22144.87	0.02	28	71.54	1026.63
	(4)	**22141**	**22141**	**22141**	**0.00**	**30**	110.13	1212.83
ch150	(1)	**6528**	6564	6532.83	0.07	19	173.23	1967.42
	(2)	**6528**	6564	6534.70	0.10	21	125.00	1484.35
	(3)	**6528**	6566	6535.17	0.11	20	**113.74**	**1011.45**
	(4)	**6528**	**6549**	**6530.77**	**0.04**	**26**	131.33	1327.90
si175	(1)	**21407**	21421	21409.57	0.01	15	102.82	3387.24
	(2)	**21407**	21420	**21408.00**	**0.01**	17	180.00	3324.80
	(3)	**21407**	21427	21410.43	0.02	17	115.20	1204.38
	(4)	**21407**	**21419**	21410.89	**0.01**	**20**	171.00	**1067.30**
pr226	(1)	**80369**	80442	80381.00	**0.01**	27	83.00	5726.51
	(2)	**80369**	80442	80421.00	**0.01**	28	83.00	3512.51
	(3)	**80369**	80440	80371.37	**0.00**	**29**	**79.45**	2000.05
	(4)	**80369**	**80373**	**80369.13**	**0.00**	**29**	92.55	**1573.50**
pcb442	(1)	**50778**	51210	50950.00	0.10	18	135.00	5200.00
	(2)	**50778**	51150	50910.00	0.10	21	120.00	4800.00
	(3)	**50778**	51180	50930.00	0.07	23	110.00	6100.00
	(4)	**50778**	**50980**	**50850.00**	**0.06**	**27**	**95.00**	**4200.00**

Across all tested TSPLIB instances, the four algorithmic configurations exhibit comparable performance on small-scale problems, consistently reaching the best-known solution in all 30 independent runs. In this regime, performance differences are mainly reflected in convergence speed rather than solution quality. For example, on `bays29` and `swiss42`, all configurations achieve zero average gap and full BKSH rates, with minor variations in the average number of iterations and runtime. However, as the size of the instance increases, clear distinctions emerge in terms of convergence dynamics and robustness. In particular, configurations incorporating adaptive learning mechanisms have more consistent convergence and reduced sensitivity to stochastic variability. This effect becomes evident from the lower average gaps, higher BKSH counts, and, in several cases, fewer iterations required to reach optimal solutions.

Contrary to the common expectation that learning-based components necessarily increase computational cost, the results indicate that adaptive mechanisms often lead to faster convergence in practice. For most tested instances, configurations that incorporate MARL initialization and/or RL-guided GVNS achieve comparable or lower intensification times while maintaining superior solution quality. This behavior can be attributed to more informed search trajectories, which reduce unnecessary exploration and accelerate the achievement of high-quality solutions.

An exception is observed on `kroB100`, where the MARL + RL-GVNS configuration incurs a higher computational time (1 213 seconds) compared to its non-adaptive counterpart (503 seconds). However, this additional cost is accompanied by a marked improvement in robustness, as the adaptive variant is the only configuration to consistently reach the best-known solution in all 30 runs, yielding a zero average gap. In contrast, faster non-adaptive configurations exhibit residual gaps and lower BKSH rates. A similar pattern is observed on larger instances, such as `ch150` and `si175`, although with distinct trade-offs. For `ch150`, MARL-based configurations achieve lower average gaps (down to 0.04%) while also showing reduced intensification times and higher best-known solution hit rates, indicating that adaptive mechanisms improve both efficiency and solution quality. The same pattern is observed in `si175`.

The time-to-target analysis complements Table 4 by characterizing convergence speed and reliability in addition to final solution quality.



(a) `berlin52` - All Combinations **(b)** `ch150` - All Combinations

Fig. 1. Time-to-target Empirical Cumulative Distribution Functions (ECDF) for representative TSPLIB instances.

As shown in Figures 1a and 1b, all configurations eventually reach the best-known solution; however, the ECDFs reveal clear differences in the convergence dynamics. Learning-based configurations, particularly under MARL initialization, exhibit steeper early curves and a noticeable leftward shift, indicating a higher probability of reaching the target solution at earlier stages. In contrast, RCL-based variants show more gradual convergence, even when combined with learning. This effect becomes more pronounced in harder instances, where adaptive configurations converge faster and more consistently, in agreement with the lower average gaps and higher BKSH rates reported in Table 4. Overall, the results indicate that reinforcement learning primarily improves convergence through more effective intensification, with computational overhead becoming noticeable only for the most challenging cases.

5 Conclusion

This work investigated the integration of reinforcement learning into a trajectory-based neighborhood search framework for the TSP, addressing limitations associated with static operator ordering in classical GVNS. To this end, an RL-GVNS framework was proposed, combining a MARL-based initialization strategy with an RL-guided local search that adaptively selects and sequences neighborhood operators during intensification.

Computational experiments on a diverse set of TSPLIB instances demonstrate that the proposed approach preserves the strong performance of classical GVNS on small-scale problems, while yielding increasingly pronounced benefits as instance size and difficulty grow. Across the evaluated instances, adaptive configurations consistently attained lower average gaps with respect to best-known solutions, higher hit rates, and more stable convergence behavior over independent runs. RL-based variants also reached high-quality solutions in fewer iterations in several cases, with reduced execution times observed for a subset of instances. When additional computational effort was incurred, it was typically associated with the most challenging instances and was accompanied by improved solution quality. Overall, the experimental results indicate that reinforcement learning can support adaptive operator selection within GVNS, contributing to improved search performance while preserving the trajectory-based structure of the method.

Future research may investigate alternative state representations and reward models in order to examine their impact on learning dynamics. Extending the proposed framework to other combinatorial optimization problems and neighborhood-based metaheuristics would allow its applicability to be acessed beyond the TSP. In addition, hybrid learning strategies combining reinforcement learning with offline training or meta-learning may be explored as a means of reducing online learning requirements.

Acknowledgments

This work is partially supported by CAPES (Grant number #001) and FAPES (#2026-6NX7B).

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used Gemini (Google) to assist with language refinement and improve the readability of the manuscript. The use of this tool was limited to editing support in the writing process. After using this service, the authors carefully reviewed and edited the content as needed and assume full responsibility for the accuracy, integrity, and originality of the final work.

References

1. Alipour, M.M., Razavi, S.N.: A new multiagent reinforcement learning algorithm to solve the symmetric traveling salesman problem. Multiagent and Grid Systems **11**(2), 107–119 (2015)
2. Alipour, M.M., Razavi, S.N., Feizi Derakhshi, M.R., Balafar, M.A.: A hybrid algorithm using a genetic algorithm and multiagent reinforcement learning heuristic to solve the traveling salesman problem. Neural Computing and Applications **30**(9), 2935–2951 (Nov 2018)
3. Babin, G., Deneault, S., Laporte, G.: Improvements to the or-opt heuristic for the symmetric travelling salesman problem. Journal of the Operational Research Society **58**(3), 402–407 (2007)
4. Basu, S., Ghosh, D.: A review of the tabu search literature on traveling salesman problems. IIMA Working Paper WP2008-10-01, Indian Institute of Management Ahmedabad, Research and Publication Department (2008)
5. Bentley, J.L.: Fast algorithms for geometric traveling salesman problems. ORSA Journal on Computing **4**(4), 387–411 (1992)
6. Blazinskas, A., Misevicius, A.: Combining 2-opt, 3-opt and 4-opt with k-swap-kick perturbations for the traveling salesman problem. In: Proceedings of the 17th International Conference on Information and Software Technologies (IT 2011). pp. 1–6. Kaunas, Lithuania (2011)
7. da Silva, R.F., Urrutia, S.: A general vns heuristic for the traveling salesman problem with time windows. Discrete Optimization **7**(4), 203–211 (2010)
8. Feo, T.A., Resende, M.G.C.: Greedy randomized adaptive search procedures. Journal of Global Optimization **6**(2), 109–133 (1995)
9. Fialho, A., Da Costa, L., Schoenauer, M., Sebag, M.: Analyzing bandit-based adaptive operator selection mechanisms. Annals of Mathematics and Artificial Intelligence **60**(1), 25–64 (2010)
10. Glover, F., Rego, C.: Local search and metaheuristics for the traveling salesman problem. In: Gutin, G., Punnen, A. (eds.) The Traveling Salesman Problem and Its Variations, pp. 309–368. Kluwer Academic Publishers, Boston (2002)
11. Hansen, P., Mladenović, N., Moreno Pérez, J.A.: Variable neighbourhood search: methods and applications. 4OR **6**(4), 319–360 (2008)
12. Hansen, P., Mladenović, N., Todosijević, R., Hanafi, S.: Variable neighborhood search: basics and variants. EURO Journal on Computational Optimization **5**(3), 423–454 (2017)
13. Hoos, H.H., Stützle, T.: Stochastic local search. In: Handbook of approximation algorithms and metaheuristics, pp. 297–307. Chapman and Hall/CRC (2018)
14. Hore, S., Chatterjee, A., Dewanji, A.: Improving variable neighborhood search to solve the traveling salesman problem. Applied Soft Computing **68**, 83–91 (2018)
15. Jeong, H.Y., Song, B.D.: Meta-learning-based adaptive operator selection for traveling salesman problem. Applied Soft Computing **185**, 113930 (2025)
16. Karakostas, P., Panoskaltsis, N., Mantalaris, A., Georgiadis, M.C.: Optimization of car t-cell therapies supply chains. Computers & Chemical Engineering **139**, 106913 (2020)
17. Karakostas, P., Sifaleras, A.: A double-adaptive general variable neighborhood search algorithm for the solution of the traveling salesman problem. Applied Soft Computing **121**, 108746 (2022)
18. Karakostas, P., Sifaleras, A., Georgiadis, M.C.: A general variable neighborhood search-based solution approach for the location-inventory-routing problem with distribution outsourcing. Computers & Chemical Engineering **126**, 263–279 (2019)

19. Karimi-Mamaghan, M., Mohammadi, M., Meyer, P., Karimi-Mamaghan, A.M., Talbi, E.G.: Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: A state-of-the-art. European Journal of Operational Research **296**(2), 393–422 (2022)
20. Lawler, E.L., Lenstra, J.K., Rinnooy Kan, A.H.G., Shmoys, D.B.: The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization. Wiley, Chichester (1985)
21. Marx, D.: Searching the k-change neighborhood for tsp is w[1]-hard. Operations Research Letters **36**(1), 31–36 (2008)
22. Menéndez, B., Bustillo, M., Pardo, E.G., Duarte, A.: General variable neighborhood search for the order batching and sequencing problem. European Journal of Operational Research **263**(1), 82–93 (2017)
23. Mladenović, N., Hansen, P.: Variable neighborhood search. Computers and Operations Research **24**(11), 1097–1100 (1997), citeSeerX: 10.1.1.800.1797
24. Osman, I.H., Kelly, J.P.: Meta-heuristics: An overview. In: Osman, I.H., Kelly, J.P. (eds.) Meta-Heuristics: Theory and Applications, pp. 1–21. Kluwer Academic Publishers, Boston, MA, USA (1996)
25. Papalitsas, C., Karakostas, P., Andronikos, T.: A performance study of the impact of different perturbation methods on the efficiency of gvns for solving tsp. Applied System Innovation **2**, 31 (09 2019)
26. Perttunen, J.: On the significance of the initial solution in travelling salesman heuristics. Journal of the Operational Research Society **45**(10), 1131–1140 (Oct 1994)
27. Queiroz dos Santos, J.P., de Melo, J.D., Duarte Neto, A.D., Aloise, D.: Reactive search strategies using reinforcement learning, local search algorithms and variable neighborhood search. Expert Systems with Applications **41**(10), 4939–4949 (2014)
28. Ruan, Y., Cai, W., Wang, J.: Combining reinforcement learning algorithm and genetic algorithm to solve the traveling salesman problem. The Journal of Engineering (2024)
29. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction. Adaptive Computation and Machine Learning, MIT Press, Cambridge, MA (1998)
30. Taillard, E., Badeau, P., Gendreau, M., Guertin, F., Potvin, J.Y.: A tabu search heuristic for the vehicle routing problem with soft time windows. Transportation Science **31**(2), 170–186 (1997)
31. Todosijević, R., Mladenović, M., Hanafi, S., Mladenović, N., Crévits, I.: Adaptive general variable neighborhood search heuristics for solving the unit commitment problem. International Journal of Electrical Power & Energy Systems **78**, 873–883 (2016)
32. Venkatesh, P., Srivastava, G., Singh, A.: A general variable neighborhood search algorithm for the k-traveling salesman problem. Procedia Computer Science **143**, 189–196 (2018), 8th International Conference on Advances in Computing & Communications (ICACC-2018)
33. Watkins, C.J.C.H.: Learning from Delayed Rewards. Ph.D. thesis, University of Cambridge, Cambridge, UK (1989)
34. Wauters, T., Verbeeck, K., De Causmaecker, P., Vanden Berghe, G.: Boosting Metaheuristic Search Using Reinforcement Learning, pp. 433–452. Springer Berlin Heidelberg, Berlin, Heidelberg (2013)